

C# / .NET — Tập 1: Cơ bản

.NET 10 LTS · C# 14 · cho người biết TypeScript/Java

2026-09-28

Mục lục

Tập 1 — C# Cơ bản

Chương 1 — Bắt đầu với .NET 10 và C# 14

Chương 2 — Biến, kiểu dữ liệu và chuỗi

Chương 3 — Điều kiện, vòng lặp và phương thức

Chương 4 — Lập trình hướng đối tượng

Chương 5 — Collections

Chương 6 — Exceptions và làm việc với file

Chương 7 — C# cho Java và TypeScript developer

GLOSSARY — Thuật ngữ (Tiếng Anh → Tiếng Việt)

Tập 1 — C# Cơ bản

Tập này dành cho người mới với C#, nhưng đã biết lập trình (TypeScript, Java). Học xong, bạn viết được chương trình console có class, collection, xử lý lỗi và đọc/ghi file.

Mục lục

#	Chương	Ý chính	Ví dụ chạy được
1	Bắt đầu với .NET 10 và C# 14	SDK, dotnet new, dotnet run, top-level statements, file-based app	Ch01.Hello
2	Biến, kiểu dữ liệu và chuỗi	value/reference type, var, số, string, nullable	Ch02.Types, Ch02.Strings
3	Điều kiện, vòng lặp, phương thức	if, switch, loops, method, ref/out, tham số mặc định	Ch03.Flow
4	Lập trình hướng đối tượng	class, property, field, constructor, interface, kế thừa, struct	Ch04.Oop
5	Collections	array, List<T>, Dictionary, HashSet, collection expressions	Ch05.Collections
6	Exceptions và file	try/catch/finally, custom exception, using, File, Path, stream	Ch06.Files
7	C# cho Java và TypeScript developer	so sánh song song cú pháp, kiểu, async, null, module	Ch07.Compare

Chạy ví dụ

```
cd books/csharp
./vol1-basics/run-examples.sh
```

Quy ước trình bày

- Thuật ngữ tiếng Anh giữ nguyên, giải thích tiếng Việt ở lần đầu; xem thêm Glossary cuối sách.
- Code và output trong sách được chép tự động từ file thật trong `examples/` sau mỗi lần chạy.
- Kiểm tra hiển thị tiếng Việt: ă ằ ẳ ẵ ơ ư đ ư Đ (nếu các chữ này hiện đúng, font đã đủ dấu).

Chương 1 — Bắt đầu với .NET 10 và C# 14

Mục tiêu

Sau chương này, bạn sẽ:

- Hiểu .NET, SDK, runtime và C# là gì, khác nhau thế nào.
- Tạo, build và chạy một chương trình console bằng `dotnet CLI`.
- Biết *top-level statements* (câu lệnh cấp cao nhất) và *file-based app* (ứng dụng một file) mới của .NET 10.

Giải thích đơn giản

Nếu bạn biết TypeScript, hãy nghĩ như sau:

TypeScript / Node	C# / .NET	Ý nghĩa
Node.js (runtime)	.NET Runtime	Chương trình chạy code của bạn
<code>tsc + npm</code>	.NET SDK (<code>dotnet CLI</code>)	Công cụ biên dịch (compile), tải gói, chạy test
<code>package.json</code>	file <code>.csproj</code>	Mô tả project và gói phụ thuộc
npm registry	NuGet (nuget.org)	Kho gói thư viện
TypeScript	C#	Ngôn ngữ lập trình

- **.NET** là nền tảng (platform) mã nguồn mở của Microsoft, chạy trên Windows, Linux, macOS.
- **C#** là ngôn ngữ chính trên .NET. Code C# được biên dịch thành *IL* (Intermediate Language — ngôn ngữ trung gian), rồi runtime dịch IL sang mã máy lúc chạy bằng *JIT* (Just-In-Time compiler). Giống Java: `.java` → bytecode → JVM.
- **SDK** (Software Development Kit) = bộ công cụ để viết code. **Runtime** = để chạy code.

Bộ sách dùng **.NET 10 (LTS)**, SDK **10.0.112**, runtime **10.0.12** và **C# 14** (chi tiết và nguồn trong [STACK.m](#)
[d](#), checked 2026-09-28).

Ví dụ

Bước 1 — Kiểm tra SDK

```
$ dotnet --version
10.0.112
```

Trong thư mục `books/csharp/` có file `global.json` ghim SDK. Nếu máy bạn có SDK 10.0.x mới hơn (ví dụ 10.0.401), `rollForward: latestFeature` vẫn cho phép dùng.

Bước 2 — Tạo project

```
$ dotnet new console -n Ch01.Hello
$ cd Ch01.Hello
$ dotnet run
```

Lệnh `dotnet new console` tạo 2 file: `Ch01.Hello.csproj` và `Program.cs`. Trong bộ sách, các thiết lập chung (TargetFramework, Nullable...) nằm ở `Directory.Build.props`, nên file `.csproj` rất ngắn:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
  </PropertyGroup>

</Project>
```

Bước 3 — Viết code

```
// Top-level statements: không cần class Program hay hàm Main.
Console.WriteLine("Xin chào, C#!");

string name = args.Length > 0 ? args[0] : "Nobin";
int year = DateTime.Now.Year;
Console.WriteLine($"Chào {name}. Năm nay là {year}.");

// Thông tin môi trường chạy
Console.WriteLine($".NET runtime: {Environment.Version}");
Console.WriteLine($"OS: {Environment.OSVersion.Platform}");
Console.WriteLine($"C# ví dụ tính toán: 7 / 2 = {7 / 2}, 7 / 2.0 = {7 / 2.0}");
```

Output thật khi chạy `dotnet run`:

```
Xin chào, C#!
Chào Nobin. Năm nay là 2026.
.NET runtime: 10.0.12
OS: Unix
C# ví dụ tính toán: 7 / 2 = 3, 7 / 2.0 = 3.5
```

Chú ý dòng cuối: `7 / 2 = 3`. Chia hai số nguyên (`int`) thì kết quả là số nguyên. Trong TypeScript, `7 / 2` là `3.5` vì mọi số đều là `number`.

Bước 4 — File-based app (mới trong .NET 10)

Từ .NET 10, bạn có thể chạy **một file .cs duy nhất**, không cần `.csproj`. Rất tiện để thử nhanh, giống `node script.js` hoặc `java Hello.java`.

```
#!/usr/bin/env dotnet
// File-based app (.NET 10): chạy trực tiếp bằng `dotnet run hello.cs`, không cần .csproj.
#:property Nullable=enable

var numbers = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
Console.WriteLine($"Có {numbers.Length} số, tổng = {numbers.Sum()}, lớn nhất = {numbers.Max()}");
```

```
$ dotnet run hello.cs
Có 8 số, tổng = 31, lớn nhất = 9
```

Dòng `#:property ...` là *directive* (chỉ thị) cho công cụ build, ngôn ngữ C# bỏ qua nó. Có các directive: `#:package` (thêm gói NuGet), `#:property`, `#:sdk`, `#:project`.

Đi sâu

Top-level statements là gì?

Trước C# 9, mọi chương trình phải có class và hàm `Main` (giống Java):

```
namespace Ch01.ClassicMain;

// Cách viết "cổ điển" trước C# 9: class Program + hàm Main.
internal class Program
{
    private static void Main(string[] args)
    {
        Console.WriteLine("Xin chào, C#! (kiểu cổ điển với Main)");
        Console.WriteLine($"Số tham số dòng lệnh: {args.Length}");
    }
}
```

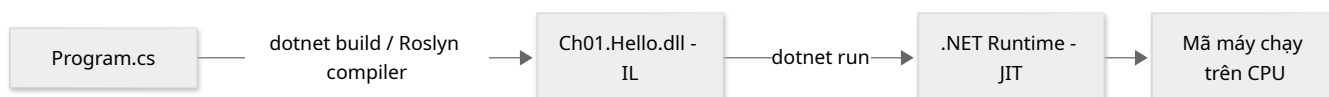
```
Xin chào, C#! (kiểu cổ điển với Main)
Số tham số dòng lệnh: 0
```

Với *top-level statements*, compiler tự sinh class và hàm `Main` cho bạn. Biến `args` (tham số dòng lệnh) vẫn dùng được. Mỗi project chỉ có **một** file được dùng top-level statements.

ImplicitUsings — vì sao không cần `using System;`?

Khi `ImplicitUsings` bật, SDK tự thêm các `using` phổ biến (`System`, `System.IO`, `System.Linq`, `System.Collections.Generic`, `System.Threading.Tasks` ...). Vì vậy `Console` dùng được ngay.

Quy trình build



- `dotnet build`: biên dịch ra `bin/Debug/net10.0/Ch01.Hello.dll`.
- `dotnet run`: build (nếu cần) rồi chạy.
- `dotnet publish -c Release`: đóng gói để triển khai (Tập 3 dùng với Docker).

Các lệnh `dotnet` hay dùng

Lệnh	Tương đương npm	Việc làm
<code>dotnet new console</code>	<code>npm init</code>	Tạo project
<code>dotnet add package X</code>	<code>npm install X</code>	Thêm gói NuGet
<code>dotnet restore</code>	<code>npm ci</code>	Tải gói
<code>dotnet build</code>	<code>tsc</code>	Biên dịch
<code>dotnet run</code>	<code>npm start</code>	Chạy
<code>dotnet test</code>	<code>npm test</code>	Chạy test

Lỗi và bất thường gặp

- **Sai SDK:** nếu thấy lỗi *"A compatible .NET SDK was not found"*, máy bạn chưa có SDK 10.0.x ≥ 10.0.112. Cài SDK .NET 10 mới nhất.
- **Chia số nguyên:** `7 / 2` là `3`, không phải `3.5`. Dùng `7 / 2.0` hoặc `(double)7 / 2`.
- **Hai file top-level:** nếu hai file cùng có top-level statements → lỗi CS8802.
- **Chạy sai thư mục:** `dotnet run` cần chạy trong thư mục có `.csproj`, hoặc dùng `dotnet run -- project <đường dẫn>`.
- **File-based app nằm trong thư mục project:** file `.cs` rời trong thư mục có `.csproj` sẽ bị compile chung vào project đó. Để file-based app ở thư mục riêng.

Tóm tắt

- .NET = runtime + thư viện; SDK = công cụ; C# = ngôn ngữ.
- `dotnet new console`, `dotnet run`, `dotnet build` là 3 lệnh đầu tiên cần nhớ.
- Top-level statements giúp `Program.cs` ngắn gọn như một script.
- .NET 10 cho phép `dotnet run file.cs` không cần project.

Bài tập (có lời giải)

Bài 1. Sửa `Program.cs` để in ra tên của bạn lấy từ tham số dòng lệnh: `dotnet run -- An` in ra `Chào An.`

▼ Lời giải

Code ở ví dụ đã đọc `args[0]`. Tham số sau `--` được truyền cho chương trình:

```
$ dotnet run -- An
Xin chào, C#!
Chào An. Năm nay là 2026.
.NET runtime: 10.0.12
OS: Unix
C# ví dụ tính toán: 7 / 2 = 3, 7 / 2.0 = 3.5
```

Dấu `--` tách tham số của `dotnet run` và tham số của chương trình.

Bài 2. Tại sao `7 / 2` in ra `3` còn `7 / 2.0` in ra `3.5`?

▼ Lời giải

`7` và `2` đều là `int`, nên phép chia là chia nguyên (bỏ phần thập phân). `2.0` là `double`, nên `7` được chuyển thành `double` và phép chia là chia số thực.

Bài 3. Viết file-based app `sum.cs` in tổng các số từ 1 đến 100.

▼ Lời giải

```
// Bài 3: tổng các số từ 1 đến 100
int sum = 0;
for (int i = 1; i <= 100; i++) sum += i;
Console.WriteLine($"Tổng 1..100 = {sum}");
```

Tổng 1..100 = 5050

Lưu code trên thành `sum.cs` ở một thư mục riêng rồi chạy `dotnet run sum.cs` (ở đây code nằm trong project `Ch01.Solutions` để script chạy tự động).

Nguồn tham khảo (Sources)

- File-based apps (nguồn Microsoft Learn): <https://github.com/dotnet/docs/blob/main/docs/core/sdk/file-based-apps.md>
- Top-level statements: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/program-structure/top-level-statements.md>
- What's new in C# 14: <https://github.com/dotnet/docs/blob/main/docs/csharp/whats-new/csharp-14.md>
- .NET 10 release data: <https://raw.githubusercontent.com/dotnet/core/main/release-notes/10.0/releases.json>

Chương 2 — Biến, kiểu dữ liệu và chuỗi

Mục tiêu

- Khai báo biến với kiểu rõ ràng và với `var`.
- Hiểu các kiểu số (`int`, `long`, `double`, `decimal`), `bool`, `char`, `string`.
- Phân biệt *value type* (kiểu giá trị) và *reference type* (kiểu tham chiếu).
- Dùng `nullable` (`int?`, `string?`) và các toán tử `?.`, `??`.
- Làm việc với chuỗi: nội suy, định dạng, `StringBuilder`, `raw string`.

Giải thích đơn giản

C# là ngôn ngữ **kiểu tĩnh** (statically typed), giống TypeScript ở chế độ `strict`, nhưng chặt hơn: kiểu tồn tại **cả lúc chạy** (runtime), không bị xóa như TypeScript.

- TypeScript chỉ có `number`. C# có nhiều kiểu số, mỗi kiểu có kích thước và độ chính xác riêng.
- `var` trong C# **không** giống `var` trong JavaScript. `var x = 5;` nghĩa là "compiler, hãy tự suy ra kiểu" → `x` là `int` mãi mãi. Giống `let x = 5` trong TS.

Kiểu C#	Kích thước	Ví dụ	Dùng cho
<code>int</code>	32 bit	42	số nguyên thông thường
<code>long</code>	64 bit	42L	số lớn, ID
<code>double</code>	64 bit	3.14	khoa học, đo lường
<code>decimal</code>	128 bit	19.99m	tiền tệ
<code>bool</code>	—	<code>true</code>	đúng/sai
<code>char</code>	16 bit	'A'	một ký tự UTF-16
<code>string</code>	—	"abc"	chuỗi (reference type, bất biến)

Ví dụ

Ví dụ 1 — Kiểu dữ liệu cơ bản

```
// 1. Khai báo biến: kiểu rõ ràng hoặc var (suy luận kiểu lúc biên dịch)
int age = 30;
var price = 19.99m;           // m = decimal (tiền tệ)
double ratio = 1.0 / 3;
bool isActive = true;
char grade = 'A';
Console.WriteLine($"age={age}, price={price}, ratio={ratio}, isActive={isActive}, grade={
grade}");
Console.WriteLine($"Kiểu của price: {price.GetType().Name}");

// 2. Giới hạn và tràn số
Console.WriteLine($"int: {int.MinValue} .. {int.MaxValue}");
int big = int.MaxValue;
int overflow = unchecked(big + 1);
Console.WriteLine($"int.MaxValue + 1 (unchecked) = {overflow}");
try
```

```

{
    int boom = checked(big + 1);
    Console.WriteLine(boom);
}
catch (OverflowException ex)
{
    Console.WriteLine($"checked: {ex.GetType().Name}");
}

// 3. double và decimal
Console.WriteLine($"0.1 + 0.2 (double) = {0.1 + 0.2}");
Console.WriteLine($"0.1 + 0.2 (decimal) = {0.1m + 0.2m}");

// 4. Chuyển kiểu
long l = age; // ngầm định (an toàn)
int back = (int)3.99; // ép kiểu: cắt phần thập phân
int parsed = int.Parse("42");
bool ok = int.TryParse("abc", out int value);
Console.WriteLine($"l={l}, (int)3.99={back}, parsed={parsed}, TryParse(\"abc\")={ok}, value={value}");

// 5. Value type vs reference type
int a = 1;
int b = a; // copy giá trị
b = 2;
int[] arr1 = [1, 2, 3];
int[] arr2 = arr1; // copy tham chiếu: cùng một mảng
arr2[0] = 99;
Console.WriteLine($"a={a}, b={b}; arr1[0]={arr1[0]} (vì arr2 trỏ cùng mảng)");

// 6. Nullable
int? maybe = null;
Console.WriteLine($"maybe.HasValue={maybe.HasValue}, maybe ?? -1 = {maybe ?? -1}");
string? nickname = null;
Console.WriteLine($"Độ dài nickname: {nickname?.Length ?? 0}");

// 7. const và hằng số
const double Pi = 3.14159;
Console.WriteLine($"Diện tích hình tròn r=2: {Pi * 2 * 2:F2}");

```

Output thật:

```

age=30, price=19.99, ratio=0.3333333333333333, isActive=True, grade=A
Kiểu của price: Decimal
int: -2147483648 .. 2147483647
int.MaxValue + 1 (unchecked) = -2147483648
checked: OverflowException
0.1 + 0.2 (double) = 0.30000000000000004
0.1 + 0.2 (decimal) = 0.3
l=30, (int)3.99=3, parsed=42, TryParse("abc")=False, value=0
a=1, b=2; arr1[0]=99 (vì arr2 trỏ cùng mảng)
maybe.HasValue=False, maybe ?? -1 = -1
Độ dài nickname: 0
Diện tích hình tròn r=2: 12.57

```

Các điểm cần để ý trong output:

- `unchecked(int.MaxValue + 1)` quay vòng về số âm. `checked(...)` ném `OverflowException`.
- `0.1 + 0.2` với `double` ra `0.30000000000000004` (giống JavaScript). Với `decimal` ra đúng `0.3`.
- `arr1[0]` bị đổi thành `99` vì `arr2 = arr1` chỉ copy **tham chiếu**.

Ví dụ 2 — Chuỗi

```
using System.Text;

string first = "Nguyễn";
string last = "Văn A";

// Nối chuỗi và nội suy chuỗi (string interpolation)
string full1 = first + " " + last;
string full2 = $"{first} {last}";
Console.WriteLine(full1 == full2); // so sánh nội dung, không phải tham chiếu

// Định dạng số trong interpolation
decimal total = 1234567.891m;
Console.WriteLine($"Tổng: {total:N2} | {total,15:F1} | {0.256:P1}");

// Các phương thức hay dùng
string s = " Hello, C# World ";
Console.WriteLine($"[{s.Trim()}] upper={s.Trim().ToUpper()} len={s.Length}");
Console.WriteLine($"Contains C#: {s.Contains("C#")}, IndexOf World: {s.IndexOf("World")}");
Console.WriteLine($"Replace: {s.Trim().Replace("World", "Việt Nam")}");
string[] parts = "a,b,c".Split(',', StringSplitOptions.RemoveEmptyEntries);
Console.WriteLine($"Split: {parts.Length} phần -> {string.Join(" | ", parts)}");

// So sánh không phân biệt hoa thường
Console.WriteLine(string.Equals("abc", "ABC", StringComparison.OrdinalIgnoreCase));

// Chuỗi là bất biến (immutable): mỗi lần "sửa" tạo chuỗi mới
string original = "abc";
string upper = original.ToUpper();
Console.WriteLine($"original={original}, upper={upper}");

// StringBuilder khi nối nhiều lần
var sb = new StringBuilder();
for (int i = 1; i <= 5; i++)
{
    sb.Append(i).Append(i < 5 ? "-" : "");
}
Console.WriteLine($"StringBuilder: {sb}");

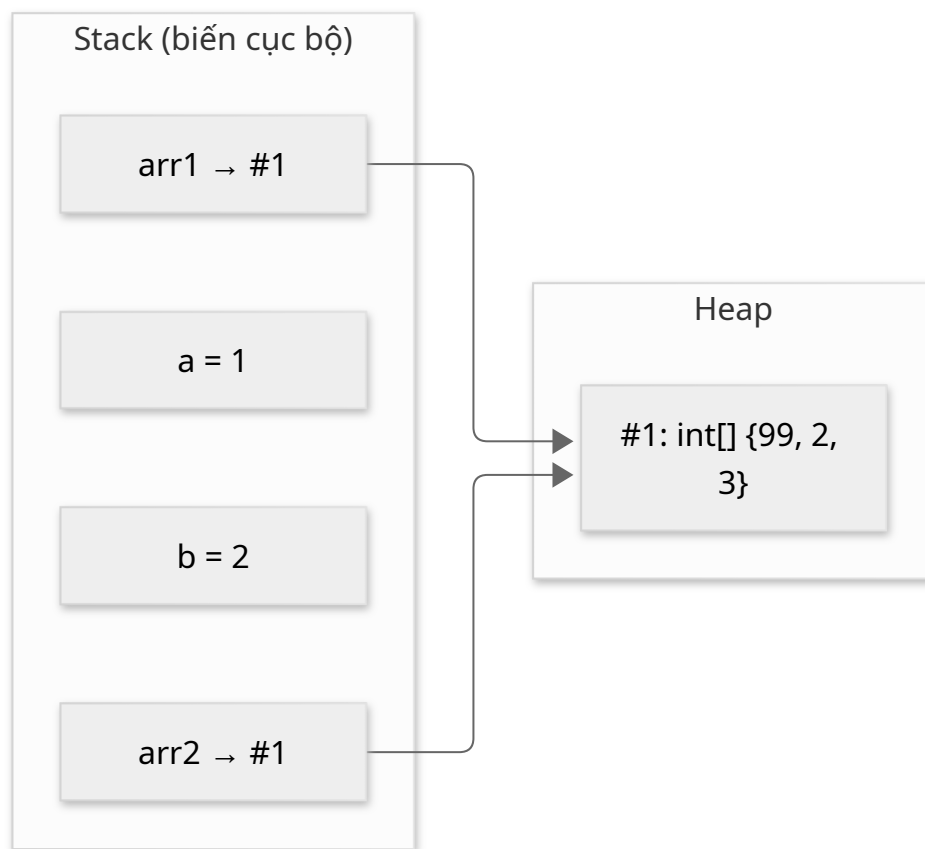
// Verbatim và raw string literal
string path = @"C:\temp\file.txt";
string json = """
    { "name": "Nobin", "lang": "C#" }
    """;
Console.WriteLine(path);
Console.WriteLine(json);

// Truy cập ký tự, index từ cuối (^1) và range (..)
string word = "TypeScript";
Console.WriteLine($"{{word[0]}} {{word[^1]}} {{word[..4]}} {{word[4..]}}");
```

```
True
Tổng: 1,234,567.89 |          1234567.9 | 25.6 %
[Hello, C# World] upper=HELLO, C# WORLD len=19
Contains C#: True, IndexOf World: 12
Replace: Hello, C# Việt Nam
Split: 3 phần -> a | b | c
True
original=abc, upper=ABC
StringBuilder: 1-2-3-4-5
C:\temp\file.txt
{ "name": "Nobin", "lang": "C#" }
T t Type Script
```

Đi sâu

Value type và reference type



- **Value type:** `int`, `double`, `decimal`, `bool`, `char`, `struct`, `enum`. Gán = copy toàn bộ giá trị.
- **Reference type:** `string`, mảng, `class`, `record (class)`, `interface`, `delegate`. Gán = copy địa chỉ; hai biến cùng trỏ một object.
- `string` là reference type nhưng **bất biến** (immutable) và so sánh `==` theo nội dung, nên dùng giống value type.

Trong Java cũng vậy: `int` là primitive, `Integer`/`String` là object. Khác biệt: C# cho phép bạn tự định nghĩa value type bằng `struct` (Chương 4).

Nullable

- `int?` là viết tắt của `Nullable<int>`: value type có thể mang giá trị `null`.
- `string?` là *nullable reference type*: chỉ là **chú thích cho compiler**. Khi `Nullable = enable` (bộ sách bật sẵn), compiler cảnh báo nếu bạn dùng biến có thể null mà không kiểm tra. Vì `TreatWarningsAsErrors = true`, cảnh báo thành lỗi build.
- `x?.Length`: nếu `x` là null thì cả biểu thức là null, không ném lỗi.
- `a ?? b`: nếu `a` null thì dùng `b`. Giống hệt TypeScript.

Chuyển kiểu

Cách	Ví dụ	Khi nào lỗi
Ngầm định (implicit)	<code>long l = someInt;</code>	không bao giờ (an toàn)

Cách	Ví dụ	Khi nào lỗi
Ép kiểu (cast)	<code>(int)3.99 → 3</code>	có thể mất dữ liệu
Parse	<code>int.Parse("42")</code>	ném <code>FormatException</code> nếu sai
TryParse	<code>int.TryParse(s, out var n)</code>	không ném, trả <code>false</code>

Với dữ liệu người dùng nhập, **luôn dùng TryParse**.

Định dạng chuỗi

{value:format} và {value,width} trong chuỗi nội suy: N2 (phân cách hàng nghìn, 2 số lẻ), F1 (1 số lẻ), P1 (phần trăm), ,15 (căn phải 15 ký tự), ,-7 (căn trái 7 ký tự). Output dùng dấu , và . kiểu Mỹ vì bộ sách bật `InvariantGlobalization` (xem `STACK.md`).

Lỗi và bẫy thường gặp

- **Dùng double cho tiền:** sai số làm tròn. Dùng `decimal`.
- **Tràn số âm thầm:** mặc định C# không kiểm tra tràn `int`. Dùng `checked` hoặc `long`.
- **So sánh chuỗi không phân biệt hoa thường** bằng `ToLower()` → tạo chuỗi mới và có thể sai với một số ngôn ngữ. Dùng `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)`.
- **Nối chuỗi trong vòng lặp lớn** bằng `+=` → chậm. Dùng `StringBuilder`.
- **Quên m:** `decimal price = 19.99`; là lỗi biên dịch, vì 19.99 là `double`. Viết `19.99m`.
- **char khác string:** `'A'` là `char`, `"A"` là `string`.

Tóm tắt

- C# có nhiều kiểu số; chọn `int` cho số đếm, `decimal` cho tiền, `double` cho đo lường.
- `var` = suy luận kiểu lúc biên dịch, kiểu không đổi.
- Value type copy giá trị; reference type copy tham chiếu.
- `?.`, `??` và `T?` giúp xử lý null an toàn.
- Chuỗi bất biến; dùng `$"..."` để nội suy và `StringBuilder` khi nối nhiều.

Bài tập (có lời giải)

1. Đổi 36.6 độ C sang độ F ($F = C * 9 / 5 + 32$), in 1 chữ số thập phân.
2. Tính tổng tiền 3 sản phẩm giá 19.990 VND, VAT 8%. Chọn kiểu dữ liệu đúng.
3. Đọc tuổi từ các chuỗi "25", "hai mươi", "" mà không để chương trình bị lỗi.
4. Đảo ngược chuỗi "Lap trinh CSharp" và đếm số nguyên âm (a, e, i, o, u).

▼ Lời giải

```
// Bài 1: đổi nhiệt độ C -> F, làm tròn 1 chữ số
double celsius = 36.6;
double fahrenheit = celsius * 9 / 5 + 32;
Console.WriteLine($"Bài 1: {celsius}°C = {fahrenheit:F1}°F");

// Bài 2: tính tiền với decimal (không dùng double cho tiền)
decimal unitPrice = 19_990m;
int quantity = 3;
```

```

decimal vat = 0.08m;
decimal total = unitPrice * quantity * (1 + vat);
Console.WriteLine($"Bài 2: tổng tiền = {total:N0} VND");

// Bài 3: đọc số an toàn bằng TryParse
foreach (var input in new[] { "25", "hai mươi", "" })
{
    string result = int.TryParse(input, out int age) ? $"tuổi = {age}" : "không hợp lệ";
    Console.WriteLine($"Bài 3: '{input}' -> {result}");
}

// Bài 4: đảo ngược chuỗi và đếm nguyên âm
string text = "Lap trình CSharp";
char[] chars = text.ToCharArray();
Array.Reverse(chars);
int vowels = text.ToLower().Count(c => "aeiou".Contains(c));
Console.WriteLine($"Bài 4: đảo ngược = '{new string(chars)}', số nguyên âm = {vowels}");

```

Bài 1: 36.6°C = 97.9°F
 Bài 2: tổng tiền = 64,768 VND
 Bài 3: '25' -> tuổi = 25
 Bài 3: 'hai mươi' -> không hợp lệ
 Bài 3: '' -> không hợp lệ
 Bài 4: đảo ngược = 'prahSC hnirt paL', số nguyên âm = 3

Giải thích: bài 2 dùng `decimal` vì là tiền; bài 3 dùng `TryParse` để không ném exception; bài 4 dùng `Count` của LINQ (sẽ học kỹ ở Tập 2).

Nguồn tham khảo (Sources)

- Built-in types: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/built-in-types/built-in-types.md>
- Integral numeric types: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/built-in-types/integral-numeric-types.md>
- Floating-point numeric types: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/built-in-types/floating-point-numeric-types.md>
- Nullable value types: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/built-in-types/nullable-value-types.md>
- Nullable reference types: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/null-safety/nullable-reference-types.md>
- Strings: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/strings/index.md>
- Raw string literals: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/string/raw-string-literals.md>

Chương 3 — Điều kiện, vòng lặp và phương thức

Mục tiêu

- Viết `if`, `switch` statement và `switch expression`.
- Dùng các vòng lặp `for`, `while`, `do-while`, `foreach`, cùng `break` và `continue`.
- Viết phương thức (method): tham số mặc định, *named arguments*, `params`, `ref`, `out`.
- Trả nhiều giá trị bằng *tuple*.

Giải thích đơn giản

Phần lớn cú pháp điều khiển của C# **giống hệt** TypeScript và Java: `if`, `for`, `while`, `break`, `continue`, `return`. Những điểm mới đáng học:

- **switch expression**: `x switch { ... }` trả về một giá trị, ngắn hơn `switch` truyền thống.
- **foreach**: giống `for...of` trong TypeScript.
- **ref / out**: truyền biến theo tham chiếu, hàm có thể sửa biến của người gọi. TypeScript và Java không có.
- **Tuple có tên**: `(int Min, int Max)` — trả nhiều giá trị mà không cần tạo class.

Ví dụ

```
// if / else
int score = 82;
if (score >= 90) Console.WriteLine("Xuất sắc");
else if (score >= 80) Console.WriteLine("Giỏi");
else Console.WriteLine("Cố gắng thêm");

// switch statement
string day = "Sat";
switch (day)
{
    case "Sat":
    case "Sun":
        Console.WriteLine("Cuối tuần");
        break;
    default:
        Console.WriteLine("Ngày làm việc");
        break;
}

// switch expression (ngắn gọn hơn)
string Rank(int s) => s switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 65 => "C",
    _ => "D",
};
Console.WriteLine($"Rank(82)={Rank(82)}, Rank(50)={Rank(50)}");

// Vòng lặp
for (int i = 0; i < 3; i++) Console.Write($"for{i} ");
Console.WriteLine();
```

```

int n = 0;
while (n < 3) { Console.Write($"while{n} "); n++; }
Console.WriteLine();

do { Console.Write("do-chạy-ít-nhất-1-lần "); } while (false);
Console.WriteLine();

foreach (var fruit in new[] { "táo", "cam", "xoài" })
{
    if (fruit == "cam") continue;    // bỏ qua phần tử này
    Console.Write($"{fruit} ");
}
Console.WriteLine();

// Gọi phương thức (local functions ở dưới cùng file)
Console.WriteLine($"Add(2, 3) = {Add(2, 3)}");
Console.WriteLine($"Greet() = {Greet()}, Greet(\"An\", \"Hi\") = {Greet(\"An\", \"Hi\")}");
Console.WriteLine($"Named args: {Greet(greeting: \"Chào\", name: \"Bình\")}");
Console.WriteLine($"params: Sum(1,2,3,4) = {Sum(1, 2, 3, 4)}");

// ref và out
int counter = 10;
Increase(ref counter);
Console.WriteLine($"Sau Increase(ref): {counter}");

if (TryDivide(10, 3, out int q, out int r))
    Console.WriteLine($"10 / 3 = {q} dư {r}");
if (!TryDivide(1, 0, out _, out _))
    Console.WriteLine("Không chia được cho 0");

// Tuple: trả về nhiều giá trị
var (min, max) = MinMax([4, 8, 1, 9]);
Console.WriteLine($"min={min}, max={max}");

// Đệ quy
Console.WriteLine($"Factorial(10) = {Factorial(10)}");

static int Add(int a, int b) => a + b;

static string Greet(string name = "bạn", string greeting = "Xin chào") => $"{greeting}, {name}!";

static int Sum(params int[] values)
{
    int total = 0;
    foreach (var v in values) total += v;
    return total;
}

static void Increase(ref int value) => value += 5;

static bool TryDivide(int a, int b, out int quotient, out int remainder)
{
    if (b == 0) { quotient = 0; remainder = 0; return false; }
    quotient = a / b;
    remainder = a % b;
    return true;
}

static (int Min, int Max) MinMax(int[] values)
{
    int min = int.MaxValue, max = int.MinValue;
    foreach (var v in values)
    {
        if (v < min) min = v;
        if (v > max) max = v;
    }
}

```

```

        return (min, max);
    }

    static long Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);

```

Output thật:

```

Giới
Cuối tuần
Rank(82)=B, Rank(50)=D
for0 for1 for2
while0 while1 while2
do-chạy-ít-nhất-1-lần
táo xoài
Add(2, 3) = 5
Greet() = Xin chào, bạn!, Greet("An", "Hi") = Hi, An!
Named args: Chào, Bình!
params: Sum(1,2,3,4) = 10
Sau Increase(ref): 15
10 / 3 = 3 dư 1
Không chia được cho 0
min=1, max=9
Factorial(10) = 3628800

```

Đi sâu

switch expression và pattern

```

string Rank(int s) => s switch
{
    >= 90 => "A",
    >= 80 => "B",
    _ => "D", // _ = "mọi trường hợp còn lại" (discard)
};

```

- Mỗi nhánh là một *pattern* (mẫu). `>= 90` là *relational pattern*.
- Compiler kiểm tra các nhánh theo thứ tự từ trên xuống.
- Nếu thiếu `_` và không nhánh nào khớp → ném `SwitchExpressionException` lúc chạy (compiler cũng cảnh báo). Tập 2 (Chương 4) học pattern matching sâu hơn.

Local function và static

Trong `Program.cs`, các hàm khai báo ở cuối file là *local function* của top-level statements. Từ khóa `static` nghĩa là hàm **không** dùng biến bên ngoài nó → an toàn, rõ ràng hơn. Hàm được gọi trước khi khai báo vẫn được, vì compiler xử lý cả file.

Truyền tham số

Cách	Ví dụ	Hàm có sửa được biến người gọi?	Yêu cầu
mặc định (by value)	Add(a, b)	không	—
ref	Increase(ref x)	có	biến phải được gán trước

Cách	Ví dụ	Hàm có sửa được biến người gọi?	Yêu cầu
out	TryDivide(10, 3, out q, out r)	có (phải gán)	hàm bắt buộc gán trước khi return
in	Print(in bigStruct)	không (chỉ đọc)	dùng cho struct lớn

Mẫu `bool TryXxx(..., out T result)` rất phổ biến trong .NET: `int.TryParse`, `Dictionary.TryGetValue`. Dùng `out _` để bỏ qua giá trị không cần.

Tham số mặc định, named arguments, params

- `Greet(string name = "bạn")`: tham số mặc định (giống TS).
- `Greet(greeting: "Chào", name: "Bình")`: gọi theo tên, không cần đúng thứ tự.
- `Sum(params int[] values)`: nhận số lượng tham số tùy ý (giống `...values` trong TS).

Expression-bodied member

`static int Add(int a, int b) => a + b;` là cách viết ngắn cho hàm chỉ có một biểu thức, giống arrow function trong TS.

Lỗi và bẫy thường gặp

- Quên break trong switch statement**: C# không cho "rơi" (fall through) từ case có code sang case khác → lỗi CS0163. Case rỗng thì được gộp (như "Sat" và "Sun").
- Quên gán tham số out** trong mọi nhánh → lỗi CS0177.
- = thay vì == trong if**: C# báo lỗi nếu kết quả không phải bool — an toàn hơn JS.
- Đệ quy quá sâu** → `StackOverflowException`, không bắt được, chương trình dừng.
- Sửa biến đếm trong foreach**: biến lặp của `foreach` là chỉ đọc.

Tóm tắt

- Cú pháp `if/for/while` giống TS/Java.
- Switch expression + pattern cho code ngắn và an toàn.
- `ref/out` truyền theo tham chiếu; mẫu `TryXxx(out ...)` rất phổ biến.
- Tuple `(int Min, int Max)` để trả nhiều giá trị.

Bài tập (có lời giải)

- In FizzBuzz từ 1 đến 15 bằng switch expression trên tuple `(i % 3, i % 5)`.
- Viết `IsPrime(int n)` và in các số nguyên tố nhỏ hơn 30.
- Viết hàm trả về tuple `(Total, Average)` của một mảng số.
- Viết `TryParseTime("12:30", out hour, out minute)` kiểm tra giờ hợp lệ.

▼ Lời giải

```
// Bài 1: FizzBuzz 1..15 bằng switch expression với tuple
Console.WriteLine("Bài 1: ");
```

```

for (int i = 1; i <= 15; i++)
{
    string s = (i % 3, i % 5) switch
    {
        (0, 0) => "FizzBuzz",
        (0, _) => "Fizz",
        (_, 0) => "Buzz",
        _ => i.ToString(),
    };
    Console.Write(s + (i < 15 ? " " : "\n"));
}

// Bài 2: kiểm tra số nguyên tố
Console.WriteLine($"Bài 2: số nguyên tố < 30: {string.Join(", ", Enumerable.Range(1, 29).Where(IsPrime))}");

// Bài 3: hàm trả về tuple (tổng, trung bình)
var (total, average) = Stats([4, 8, 15, 16, 23, 42]);
Console.WriteLine($"Bài 3: tổng={total}, trung bình={average:F2}");

// Bài 4: TryParse kiểu riêng với out
Console.WriteLine($"Bài 4: '12:30' hợp lệ? {TryParseTime("12:30", out var h, out var m)} -> {h}{h{m}}");
Console.WriteLine($"Bài 4: '25:99' hợp lệ? {TryParseTime("25:99", out _, out _)}");

static bool IsPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) return false;
    return true;
}

static (int Total, double Average) Stats(int[] values)
{
    int total = 0;
    foreach (var v in values) total += v;
    return (total, (double)total / values.Length);
}

static bool TryParseTime(string text, out int hour, out int minute)
{
    hour = minute = 0;
    var parts = text.Split(':');
    return parts.Length == 2
        && int.TryParse(parts[0], out hour) && hour is >= 0 and < 24
        && int.TryParse(parts[1], out minute) && minute is >= 0 and < 60;
}

```

```

Bài 1: 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
Bài 2: số nguyên tố < 30: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29
Bài 3: tổng=108, trung bình=18.00
Bài 4: '12:30' hợp lệ? True -> 12h30
Bài 4: '25:99' hợp lệ? False

```

Chú ý bài 4: `hour is >= 0 and < 24` là *pattern* kết hợp `and`, dễ đọc hơn `hour >= 0 && hour < 24`.

Nguồn tham khảo (Sources)

- Selection statements: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/statements/selection.md>

- Iteration statements: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/statements/iteration.md>
- Relational and logical patterns: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/patterns/relational-logical-patterns.md>
- Tuples: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/tuples.md>
- Discards: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/patterns/discards.md>

Chương 4 — Lập trình hướng đối tượng

Mục tiêu

- Viết class với field, *property*, constructor, phương thức.
- Dùng từ khóa `field` mới của C# 14 trong *property*.
- Hiểu *access modifier* (`public`, `private`, `protected`, `internal`).
- Dùng interface, abstract class, kế thừa, `override`, `sealed`.
- Biết khi nào dùng `struct` thay vì `class`.
- Dùng *primary constructor*, `required` và `init`.

Giải thích đơn giản

OOP trong C# rất giống Java và class trong TypeScript. Khác biệt lớn nhất là **property**:

- Java: `private int age; getAge(); setAge(...)`.
- TypeScript: `get age() {...} / set age(v) {...}`.
- C#: `public int Age { get; set; }` — một dòng, compiler tự tạo field ẩn (*backing field*).

Property nhìn như field khi dùng (`account.Balance`), nhưng thực chất là hàm `get` / `set`, nên bạn có thể thêm kiểm tra dữ liệu (*validation*) sau này mà không đổi code gọi.

Ví dụ

Project `Ch04.Oop` có 3 file.

BankAccount.cs — class với property, constructor và từ khóa `field`:

```
namespace Ch04.Oop;

// Class với field private, property, constructor và phương thức
public class BankAccount
{
    private readonly List<string> _history = [];

    public BankAccount(string owner, decimal initialBalance = 0)
    {
        Owner = owner;
        Balance = initialBalance;
        _history.Add($"Mở tài khoản: {initialBalance:N0}");
    }

    public string Owner { get; } // chỉ đọc (read-only)
    public decimal Balance { get; private set; } // chỉ class này được gán

    // C# 14: từ khóa 'field' = backing field do compiler tạo sẵn
    public string Note
    {
        get;
        set => field = string.IsNullOrEmpty(value) ? "(trống)" : value.Trim();
    } = "(trống)";

    public void Deposit(decimal amount)
    {

```

```

        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount), "Số tiền phải >
0");
        Balance += amount;
        _history.Add($"Nạp: {amount:N0}");
    }

    public bool TryWithdraw(decimal amount)
    {
        if (amount > Balance) return false;
        Balance -= amount;
        _history.Add($"Rút: {amount:N0}");
        return true;
    }

    public IReadOnlyList<string> History => _history;

    public override string ToString() => $"{Owner}: {Balance:N0} VND";
}

```

Shapes.cs — interface, abstract class, kế thừa, struct, required:

```

namespace Ch04.Oop;

// Interface: "hợp đồng" mà class phải thực hiện
public interface IShape
{
    string Name { get; }
    double Area();
    // Default interface method (có thân hàm sẵn)
    string Describe() => $"{Name} có diện tích {Area():F2}";
}

// Abstract class: không tạo object trực tiếp được
public abstract class Shape : IShape
{
    public abstract string Name { get; }
    public abstract double Area();
    public override string ToString() => ((IShape)this).Describe();
}

// Primary constructor (C# 12): tham số (width, height) dùng trong cả class
public class Rectangle(double width, double height) : Shape
{
    public double Width { get; } = width;
    public double Height { get; } = height;
    public override string Name => "Hình chữ nhật";
    public override double Area() => Width * Height;
}

public sealed class Square(double side) : Rectangle(side, side)
{
    public override string Name => "Hình vuông";
}

public class Circle(double radius) : Shape
{
    public override string Name => "Hình tròn";
    public override double Area() => Math.PI * radius * radius;
}

// struct: value type, thích hợp cho dữ liệu nhỏ, bất biến
public readonly struct Point(int x, int y)
{
    public int X { get; } = x;
    public int Y { get; } = y;
    public Point Move(int dx, int dy) => new(X + dx, Y + dy);
}

```

```

        public override string ToString() => $"({X}, {Y})";
    }

    // required + init: bắt buộc gán khi khởi tạo, không sửa được sau đó
    public class Product
    {
        public required string Sku { get; init; }
        public required string Name { get; init; }
        public decimal Price { get; init; }
        public static int CreatedCount { get; private set; } // static: thuộc về class
        public Product() => CreatedCount++;
    }

```

Program.cs:

```

using Ch04.Oop;

// 1. Class và object
var account = new BankAccount("Nobin", 1_000_000);
account.Deposit(500_000);
bool ok = account.TryWithdraw(2_000_000);
Console.WriteLine($"Rút 2 triệu thành công? {ok}");
account.TryWithdraw(300_000);
account.Note = "    tài khoản lương    ";
Console.WriteLine(account);
Console.WriteLine($"Note: '{account.Note}'");
Console.WriteLine("Lịch sử: " + string.Join("; ", account.History));

try
{
    account.Deposit(-5);
}
catch (ArgumentOutOfRangeException ex)
{
    Console.WriteLine($"Lỗi: {ex.Message}");
}

// 2. Kế thừa và đa hình (polymorphism)
List<IShape> shapes = [new Rectangle(3, 4), new Square(2), new Circle(1)];
foreach (var shape in shapes)
{
    Console.WriteLine(shape.Describe());
}
Console.WriteLine($"Square là Rectangle? {shapes[1] is Rectangle}");

// 3. struct là value type: copy khi gán
var p1 = new Point(1, 2);
var p2 = p1.Move(10, 0);
Console.WriteLine($"p1={p1}, p2={p2}");

// 4. required / init / static
var product = new Product { Sku = "SKU-1", Name = "Bàn phím", Price = 750_000 };
_ = new Product { Sku = "SKU-2", Name = "Chuột" };
Console.WriteLine($"{{product.Name}} giá {product.Price:N0}; đã tạo {Product.CreatedCount} sản phẩm");

```

Output thật:

```

Rút 2 triệu thành công? False
Nobin: 1,200,000 VND
Note: 'tài khoản lương'
Lịch sử: Mở tài khoản: 1,000,000; Nạp: 500,000; Rút: 300,000
Lỗi: Số tiền phải > 0 (Parameter 'amount')
Hình chữ nhật có diện tích 12.00
Hình vuông có diện tích 4.00

```

Hình tròn có diện tích 3.14
Square là Rectangle? True
p1=(1, 2), p2=(11, 2)
Bàn phím giá 750,000; đã tạo 2 sản phẩm

Đi sâu

Property và từ khóa `field` (C# 14)

Trước C# 14, muốn thêm logic vào `set` bạn phải tự khai báo `field`:

```
private string _note = "(trống)";  
public string Note { get => _note; set => _note = value.Trim(); }
```

Với C# 14, `field` là backing field do compiler tạo:

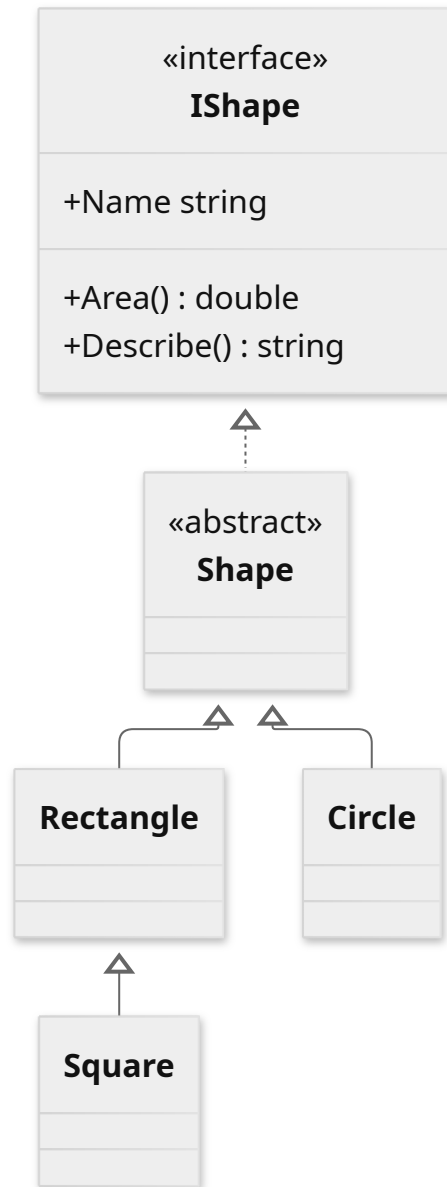
```
public string Note { get; set => field = value.Trim(); } = "(trống)";
```

Ngắn hơn và không ai trong class truy cập nhầm `_note` được. Nếu class của bạn đã có một thành viên tên `field`, dùng `@field` hoặc `this.field` để phân biệt.

Access modifiers

Modifier	AI truy cập được	Java tương đương
<code>public</code>	mọi nơi	<code>public</code>
<code>private</code>	chỉ trong class (mặc định cho member)	<code>private</code>
<code>protected</code>	class và class con	<code>protected</code> (Java còn cho cả package)
<code>internal</code>	trong cùng assembly (project) — mặc định cho class	gần giống package-private
<code>protected internal</code> , <code>private protected</code>	kết hợp	—

Kế thừa và đa hình



- C# chỉ cho kế thừa **một** class, nhưng implement **nhiều** interface (giống Java).
- Phương thức muốn cho class con ghi đè phải là `virtual` hoặc `abstract`; class con dùng `override`. Java thì mọi method mặc định là `virtual` — C# thì **không**.
- `sealed` = không cho kế thừa tiếp (giống `final` của Java).
- Interface có thể có *default method* (`Describe()` ở trên).

Primary constructor (C# 12)

`public class Circle(double radius) : Shape` — tham số `radius` dùng được trong toàn class. Nó **không** tự trở thành property; muốn public thì gán: `public double Radius { get; } = radius;`. Với `record` (Tập 2), tham số của primary constructor **tự** thành property.

struct hay class?

	class	struct
Loại	reference type	value type
Gán	copy tham chiếu	copy toàn bộ dữ liệu
Null	có thể null	không (trừ T?)
Kế thừa	có	không
Dùng khi	hầu hết trường hợp	dữ liệu nhỏ ($\leq \sim 16$ byte), bất biến: <code>Point</code> , <code>Money</code>

Nên dùng `readonly struct` để tránh lỗi copy rồi sửa nhầm bản copy.

required và init

- `init`: chỉ gán được lúc khởi tạo (`new Product { Name = ... }`), sau đó chỉ đọc.
- `required`: compiler bắt buộc người tạo object phải gán. Thiếu $\rightarrow$ lỗi CS9035.

Lỗi và bẫy thường gặp

- Quên `virtual`**: viết method cùng tên ở class con mà cha không `virtual` $\rightarrow$ đó là *ẩn* (hiding, cảnh báo CS0108), không phải override. Gọi qua biến kiểu cha sẽ chạy method của cha.
- Field public**: nên dùng property, không public field (trừ `const/static readonly`).
- Struct lớn, có thể sửa**: copy ngầm khiến bạn sửa bản copy, bản gốc không đổi.
- Gọi method virtual trong constructor**: class con chưa khởi tạo xong.
- So sánh object bằng `==`**: với class, `==` mặc định so sánh tham chiếu. Muốn so sánh theo giá trị $\rightarrow$ dùng `record` (Tập 2).

Tóm tắt

- Property = cách chuẩn để lộ dữ liệu; `field` (C# 14) giúp thêm logic dễ dàng.
- Một class cha, nhiều interface; phải `virtual/abstract` mới `override` được.
- Primary constructor, `required`, `init` giúp code ngắn và an toàn.
- `struct` cho dữ liệu nhỏ, bất biến; còn lại dùng `class`.

Bài tập (có lời giải)

- Viết class `Student` có `Name` (chỉ đọc) và `Score` (0–10, ném exception nếu sai) dùng `field`.
- Cho `Student` implement interface `IGradable` với method `Grade()` trả về xếp loại.
- Viết abstract class `Animal` với `Speak()`, và class `Dog`, `Cat` override nó.
- Viết `readonly struct Money(decimal amount, string currency)` với method `Multiply`.

▼ Lời giải

```
// Bài 1 + 2: class Student với validation bằng `field`, và interface IGradable
var s = new Student("An") { Score = 8.5 };
Console.WriteLine($"Bài 1: {s.Name} điểm {s.Score}, xếp loại {s.Grade()}");
try
{
```

```

        s.Score = 11;
    }
    catch (ArgumentOutOfRangeException ex)
    {
        Console.WriteLine($"Bài 1: lỗi -> {ex.ParamName}: {ex.Message.Split(" ")[0]}");
    }

    // Bài 3: kế thừa + override
    Animal[] animals = [new Dog("Mực"), new Cat("Mướp")];
    foreach (var a in animals) Console.WriteLine($"Bài 3: {a.Name} kêu {a.Speak()}");

    // Bài 4: struct Money bất biến
    var price = new Money(100_000, "VND");
    var doubled = price.Multiply(2);
    Console.WriteLine($"Bài 4: {price} x2 = {doubled}");

    public interface IGradable
    {
        string Grade();
    }

    public class Student(string name) : IGradable
    {
        public string Name { get; } = name;

        public double Score
        {
            get;
            set => field = value is >= 0 and <= 10
                ? value
                : throw new ArgumentOutOfRangeException(nameof(Score), "Điểm phải từ 0 đến 10");
        }

        public string Grade() => Score switch { >= 8 => "Giỏi", >= 6.5 => "Khá", >= 5 => "TB",
_ => "Yếu" };
    }

    public abstract class Animal(string name)
    {
        public string Name { get; } = name;
        public abstract string Speak();
    }

    public class Dog(string name) : Animal(name) { public override string Speak() => "Gâu gâu"; }
    public class Cat(string name) : Animal(name) { public override string Speak() => "Meo meo"; }

    public readonly struct Money(decimal amount, string currency)
    {
        public decimal Amount { get; } = amount;
        public string Currency { get; } = currency;
        public Money Multiply(decimal factor) => new(Amount * factor, Currency);
        public override string ToString() => $"{Amount:N0} {Currency}";
    }

```

Bài 1: An điểm 8.5, xếp loại Giỏi
 Bài 1: lỗi -> Score: Điểm phải từ 0 đến 10
 Bài 3: Mực kêu Gâu gâu
 Bài 3: Mướp kêu Meo meo
 Bài 4: 100,000 VND x2 = 200,000 VND

Chú ý: type declarations (class, interface, struct) phải đặt **sau** top-level statements trong cùng file.

Nguồn tham khảo (Sources)

- Classes: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/classes.md>
- Inheritance: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/object-oriented/inheritance.md>
- Polymorphism: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/object-oriented/polymorphism.md>
- Interfaces: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/interfaces.md>
- Structs: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/structs.md>
- `field` keyword (C# 14 proposal): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-14.0/field-keyword.md>
- Primary constructors (C# 12 proposal): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-12.0/primary-constructors.md>

Chương 5 — Collections

Mục tiêu

- Dùng mảng (array) một chiều và nhiều chiều.
- Dùng `List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`.
- Dùng *collection expressions* `[1, 2, 3]` và *spread* `...`.
- Biết chọn collection phù hợp và tránh lỗi sửa collection khi đang duyệt.

Giải thích đơn giản

Collection (tập hợp) là đối tượng chứa nhiều phần tử. So sánh nhanh:

Việc cần	TypeScript	Java	C#
Danh sách động	<code>Array / T[]</code>	<code>ArrayList<T></code>	<code>List<T></code>
Mảng cố định	—	<code>T[]</code>	<code>T[]</code>
Khóa → giá trị	<code>Map<K,V> / object</code>	<code>HashMap<K,V></code>	<code>Dictionary<TKey,TValue></code>
Không trùng	<code>Set<T></code>	<code>HashSet<T></code>	<code>HashSet<T></code>
Hàng đợi / ngăn xếp	tự viết	<code>ArrayDeque</code>	<code>Queue<T> / Stack<T></code>

Chữ `<T>` là *generic* — "danh sách của kiểu T". Tập 2 học generics kỹ hơn. Khác Java: C# generics giữ kiểu lúc chạy, và `List<int>` chứa `int` thật (không boxing thành `Integer`).

Ví dụ

```
// 1. Array: kích thước cố định
int[] scores = [90, 75, 60, 88];
Array.Sort(scores);
Console.WriteLine($"Array đã sắp xếp: {string.Join(", ", scores)}; Length={scores.Length}");

int[,] matrix = { { 1, 2 }, { 3, 4 } };
Console.WriteLine($"matrix[1,0]={matrix[1, 0]}");

// 2. List<T>: mảng động
List<string> names = ["An", "Bình"];
names.Add("Chi");
names.Insert(0, "Dũng");
names.Remove("Bình");
Console.WriteLine($"List: {string.Join(", ", names)} (Count={names.Count})");
Console.WriteLine($"Có 'Chi'? {names.Contains("Chi")}; vị trí của 'Chi' = {names.IndexOf("Chi")}");
names.Sort();
Console.WriteLine($"Sau Sort: {string.Join(", ", names)}");

// 3. Dictionary<TKey, TValue>: tra cứu theo khóa
var stock = new Dictionary<string, int>
{
    ["apple"] = 10,
    ["banana"] = 5,
};
stock["cherry"] = 7;           // thêm hoặc ghi đè
stock["apple"] += 3;
```

```

if (stock.TryGetValue("banana", out int bananas))
    Console.WriteLine($"banana: {bananas}");
Console.WriteLine($"Có 'durian'? {stock.ContainsKey("durian")}");
foreach (var (fruit, qty) in stock)
    Console.WriteLine($" {fruit,-7} = {qty}");

// 4. HashSet<T>: không trùng lặp, phép toán tập hợp
var backend = new HashSet<string> { "C#", "Java", "Go" };
var frontend = new HashSet<string> { "TypeScript", "C#" };
Console.WriteLine($"Add 'Java' lần 2: {backend.Add("Java")}");
var both = new HashSet<string>(backend);
both.IntersectWith(frontend);
Console.WriteLine($"Giao: {string.Join(", ", both)}");

// 5. Queue và Stack
var queue = new Queue<string>(["job1", "job2"]);
queue.Enqueue("job3");
Console.WriteLine($"Queue.Dequeue() = {queue.Dequeue()} (FIFO)");
var stack = new Stack<int>([1, 2]);
stack.Push(3);
Console.WriteLine($"Stack.Pop() = {stack.Pop()} (LIFO)");

// 6. Collection expressions và spread (...)
int[] a = [1, 2];
int[] b = [3, 4];
List<int> all = [.. a, .. b, 5];
Console.WriteLine($"Spread: [{string.Join(", ", all)}]");

// 7. Read-only views
IReadOnlyList<int> view = all.AsReadOnly();
Console.WriteLine($"Read-only view Count={view.Count}, phần tử cuối={view[^1]}");

// 8. Sửa collection khi đang foreach -> lỗi
try
{
    foreach (var n in all)
        if (n == 2) all.Remove(n);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Lỗi khi sửa trong foreach: {ex.GetType().Name}");
}
all.RemoveAll(n => n % 2 == 0); // cách đúng
Console.WriteLine($"Sau RemoveAll số chẵn: [{string.Join(", ", all)}]");

```

Output thật:

```

Array đã sắp xếp: 60, 75, 88, 90; Length=4
matrix[1,0]=3
List: Dũng, An, Chi (Count=3)
Có 'Chi'? True; vị trí của 'Chi' = 2
Sau Sort: An, Chi, Dũng
banana: 5
Có 'durian'? False
  apple  = 13
  banana = 5
  cherry = 7
Add 'Java' lần 2: False
Giao: C#
Queue.Dequeue() = job1 (FIFO)
Stack.Pop() = 3 (LIFO)
Spread: [1, 2, 3, 4, 5]
Read-only view Count=5, phần tử cuối=5
Lỗi khi sửa trong foreach: InvalidOperationException
Sau RemoveAll số chẵn: [1, 3, 5]

```

Đi sâu

Chọn collection nào?

Collection	Thêm	Tìm theo khóa/giá trị	Truy cập theo index	Thứ tự
T[]	không (cố định)	O(n)	O(1)	giữ
List<T>	O(1) trung bình ở cuối	O(n) (Contains)	O(1)	giữ
Dictionary<K,V>	O(1) trung bình	O(1) theo khóa	—	không đảm bảo
HashSet<T>	O(1) trung bình	O(1)	—	không đảm bảo
SortedDictionary<K,V>	O(log n)	O(log n)	—	sắp theo khóa
Queue<T> / Stack<T>	O(1)	O(n)	—	FIFO / LIFO

Quy tắc nhanh: cần tra cứu theo khóa → Dictionary; cần "có hay không" → HashSet; còn lại → List<T>.

Collection expressions (C# 12)

```
int[] a = [1, 2];
List<int> list = [... a, 3]; // spread: giống [...a, 3] trong TS
Span<int> span = [1, 2, 3]; // cũng dùng được với Span (Tập 3)
```

Cùng cú pháp [...] tạo được array, List<T>, HashSet<T>, Span<T> ... Kiểu đích quyết định collection nào được tạo.

Duyệt Dictionary

foreach (var (fruit, qty) in stock) dùng *deconstruction* (tách) KeyValuePair thành 2 biến. Thứ tự duyệt của Dictionary **không được đảm bảo** — đừng viết code phụ thuộc vào thứ tự.

Interface của collection

Khi viết method, nhận tham số bằng interface nhỏ nhất bạn cần:

- IEnumerable<T>: chỉ cần duyệt (foreach, LINQ).
- IReadOnlyList<T> / IReadOnlyCollection<T>: cần Count, index, nhưng không sửa.
- IList<T>, ICollection<T>: cần sửa.

Trả về IReadOnlyList<T> để người gọi không sửa dữ liệu nội bộ của bạn (như BankAccount.History ở Chương 4).

Lỗi và bẫy thường gặp

- **Sửa collection khi đang foreach** → InvalidOperationException. Dùng RemoveAll, duyệt ngược bằng for, hoặc tạo list mới.

- `dict[key]` khi key không tồn tại → `KeyNotFoundException`. Dùng `TryGetValue` hoặc `GetValueOrDefault`.
- **Tường Dictionary** giữ thứ tự thêm vào: không đảm bảo.
- **Mảng có kích thước cố định**: không có `Add`. Cần thêm/xóa → dùng `List<T>`.
- `List.Contains` trong vòng lặp lớn → $O(n^2)$. Dùng `HashSet<T>`.

Tóm tắt

- `List<T>` là collection dùng nhiều nhất; `Dictionary` để tra cứu; `HashSet` để loại trùng.
- `[ ... ]` và `..` tạo collection ngắn gọn như TypeScript.
- Trả về `ReadOnlyList<T>`/`IEnumerable<T>` để bảo vệ dữ liệu.
- Không sửa collection khi đang duyệt.

Bài tập (có lời giải)

1. Đếm tần suất mỗi từ trong câu "một hai ba hai ba ba" bằng `Dictionary`.
2. Loại phần tử trùng trong `[3, 1, 3, 2, 1, 5]`, giữ thứ tự xuất hiện đầu tiên.
3. Kiểm tra chuỗi ngoặc `"(a[b]{c})"`, `"()"`, `"(("` có cân bằng không, dùng `Stack<char>`.
4. Gộp `[5, 1]` và `[4, 9]` bằng `spread`, rồi sắp xếp giảm dần.

▼ Lời giải

```
// Bài 1: đếm tần suất từ bằng Dictionary
string text = "một hai ba hai ba ba";
var freq = new Dictionary<string, int>();
foreach (var w in text.Split(' '))
{
    freq[w] = freq.GetValueOrDefault(w) + 1;
}
foreach (var (word, count) in freq) Console.WriteLine($"Bài 1: {word} = {count}");

// Bài 2: loại bỏ phần tử trùng, giữ thứ tự xuất hiện
int[] input = [3, 1, 3, 2, 1, 5];
var seen = new HashSet<int>();
var unique = new List<int>();
foreach (var n in input)
    if (seen.Add(n)) unique.Add(n); // Add trả về false nếu đã có
Console.WriteLine($"Bài 2: [{string.Join(", ", unique)}]");

// Bài 3: kiểm tra dấu ngoặc cân bằng bằng Stack
foreach (var expr in new[] { "(a[b]{c})", "()", "(" })
    Console.WriteLine($"Bài 3: {expr,-10} cân bằng? {IsBalanced(expr)}");

// Bài 4: gộp hai mảng bằng spread và sắp xếp giảm dần
int[] a = [5, 1], b = [4, 9];
List<int> merged = [.. a, .. b];
merged.Sort((x, y) => y.CompareTo(x));
Console.WriteLine($"Bài 4: [{string.Join(", ", merged)}]");

static bool IsBalanced(string s)
{
    var pairs = new Dictionary<char, char> { [')'] = '(', ['}] = '{', ['}] = '{' };
    var stack = new Stack<char>();
    foreach (var c in s)
    {
        if (c is '(' or '[' or '{') stack.Push(c);
        else if (pairs.TryGetValue(c, out var open))
        {
            stack.Pop();
        }
    }
    return stack.Count == 0;
}
```

```

        {
            if (stack.Count == 0 || stack.Pop() != open) return false;
        }
    }
    return stack.Count == 0;
}

```

Bài 1: một = 1
 Bài 1: hai = 2
 Bài 1: ba = 3
 Bài 2: [3, 1, 2, 5]
 Bài 3: (a[b]{c}) cân bằng? True
 Bài 3: () cân bằng? False
 Bài 3: ((cân bằng? False
 Bài 4: [9, 5, 4, 1]

Bài 2 dùng mẹo: `HashSet.Add` trả về `false` nếu phần tử đã có.

Nguồn tham khảo (Sources)

- Collections (C# fundamentals): <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/statements/collections.md>
- Collections (language reference): <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/builtin-types/collections.md>
- Collection expressions: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/operators/collection-expressions.md>
- Arrays: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/builtin-types/arrays.md>
- Collection expressions proposal (C# 12): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-12.0/collection-expressions.md>

Chương 6 — Exceptions và làm việc với file

Mục tiêu

- Bắt và xử lý lỗi với `try / catch / finally`, *exception filter* (when).
- Tạo *custom exception* (exception tự định nghĩa) và bọc lỗi bằng `InnerException`.
- Dùng `using` để giải phóng tài nguyên (file, kết nối).
- Đọc/ghi file với `File`, `StreamReader`, `StreamWriter`, `Path`, `Directory`, `FileInfo`.

Giải thích đơn giản

Exception (ngoại lệ) là cách .NET báo lỗi: khi có lỗi, runtime "ném" (throw) một object exception, và chương trình nhảy đến khối `catch` gần nhất phù hợp. Giống `try/catch` trong TS và Java, với vài khác biệt:

- C# **không có checked exception** như Java. Không cần khai báo `throws`.
- Bạn bắt theo **kiểu** exception: `catch (FileNotFoundException ex)`. TS chỉ có `catch (e)`.
- `using` tự động gọi `Dispose()` để đóng file — giống `try-with-resources` của Java.

Ví dụ

Custom exception dùng primary constructor:

```
namespace Ch06.Files;

// Custom exception: kế thừa Exception, thêm dữ liệu riêng
public class InsufficientStockException(string sku, int requested, int available)
    : Exception($"Không đủ hàng cho {sku}: cần {requested}, còn {available}")
{
    public string Sku { get; } = sku;
    public int Requested { get; } = requested;
    public int Available { get; } = available;
}
```

Chương trình chính (phần 1: exceptions, phần 2: file):

```
using System.Text;
using Ch06.Files;

// ----- Phần 1: Exceptions -----
static int ParseQuantity(string input)
{
    try
    {
        return int.Parse(input);
    }
    catch (FormatException)
    {
        Console.WriteLine($" '{input}' không phải số -> dùng 0");
        return 0;
    }
    finally
    {
    }
}
```

```

        Console.WriteLine($" finally luôn chạy (input='{input}')");
    }
}

Console.WriteLine($"ParseQuantity(\"12\") = {ParseQuantity(\"12\")}");
Console.WriteLine($"ParseQuantity(\"x\") = {ParseQuantity(\"x\")}");

static void Reserve(string sku, int requested, int available)
{
    if (requested > available)
        throw new InsufficientStockException(sku, requested, available);
    Console.WriteLine($" Đã giữ {requested} {sku}");
}

try
{
    Reserve("SKU-1", 2, 5);
    Reserve("SKU-2", 9, 3);
}
catch (InsufficientStockException ex) when (ex.Available > 0) // exception filter
{
    Console.WriteLine($" Bắt được: {ex.Message} (thiếu {ex.Requested - ex.Available})");
}

// Bọc exception để thêm ngữ cảnh, giữ InnerException
try
{
    try { _ = int.Parse("abc"); }
    catch (FormatException inner) { throw new InvalidOperationException("Độc cấu hình thất bại", inner); }
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($" {ex.Message} <- {ex.InnerException?.GetType().Name}");
}

// ----- Phần 2: File và thư mục -----
string dir = Path.Combine(Path.GetTempPath(), "csharp-book-ch06");
Directory.CreateDirectory(dir);
string csvPath = Path.Combine(dir, "products.csv");

// Ghi toàn bộ file một lần
string[] lines =
[
    "sku,name,price",
    "SKU-1,Bàn phím,750000",
    "SKU-2,Chuột,250000",
    "SKU-3,Màn hình,3200000",
];
File.WriteAllLines(csvPath, lines, Encoding.UTF8);
Console.WriteLine($"Đã ghi {lines.Length} dòng vào {Path.GetFileName(csvPath)}");

// Đọc từng dòng bằng StreamReader + using (tự Dispose khi ra khỏi scope)
decimal total = 0;
using (var reader = new StreamReader(csvPath, Encoding.UTF8))
{
    _ = reader.ReadLine(); // bỏ header
    string? line;
    while ((line = reader.ReadLine()) is not null)
    {
        var cols = line.Split(',');
        total += decimal.Parse(cols[2]);
        Console.WriteLine($" {cols[0]} | {cols[1],-10} | {decimal.Parse(cols[2]),12:N0}");
    }
}
Console.WriteLine($"Tổng giá: {total:N0}");

```

```
// Ghi thêm (append) bằng using declaration (C# 8)
{
    using var writer = new StreamWriter(csvPath, append: true, Encoding.UTF8);
    writer.WriteLine("SKU-4,Tai nghe,990000");
}
Console.WriteLine($"Số dòng sau khi append: {File.ReadAllLines(csvPath).Length}");

// Thông tin file
var info = new FileInfo(csvPath);
Console.WriteLine($"Kích thước: {info.Length} bytes, đuôi: {info.Extension}");

// File không tồn tại
try
{
    File.ReadAllText(Path.Combine(dir, "missing.txt"));
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"FileNotFoundException: {Path.GetFileName(ex.FileName)}");
}

// Dọn dẹp
Directory.Delete(dir, recursive: true);
Console.WriteLine($"Đã xóa thư mục tạm? {!Directory.Exists(dir)}");
```

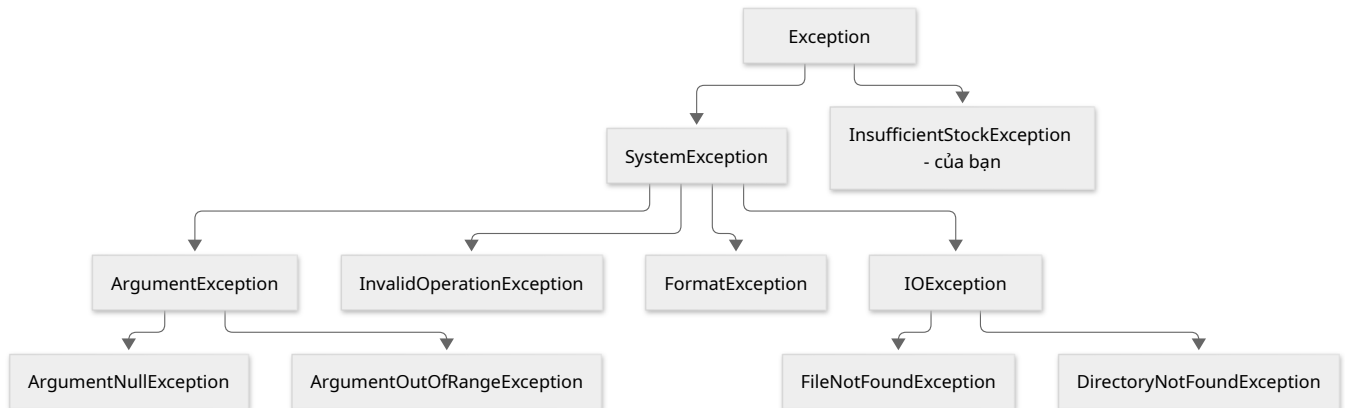
Output thật:

```
finally luôn chạy (input='12')
ParseQuantity("12") = 12
'x' không phải số -> dùng 0
finally luôn chạy (input='x')
ParseQuantity("x") = 0
Đã giữ 2 SKU-1
Bắt được: Không đủ hàng cho SKU-2: cần 9, còn 3 (thiếu 6)
Đọc cấu hình thất bại <- FormatException
Đã ghi 4 dòng vào products.csv
SKU-1 | Bàn phím | 750,000
SKU-2 | Chuột | 250,000
SKU-3 | Màn hình | 3,200,000
Tổng giá: 4,200,000
Số dòng sau khi append: 5
Kích thước: 110 bytes, đuôi: .csv
FileNotFoundException: missing.txt
Đã xóa thư mục tạm? True
```

Để ý thứ tự dòng: `finally` luôn chạy được in **trước** `ParseQuantity("12") = 12`, vì `finally` chạy trước khi hàm thực sự trả về, còn dòng `Console.WriteLine` bên ngoài chỉ chạy sau khi có kết quả.

Đi sâu

Cây exception thường gặp



- Bắt exception **cụ thể** trước, tổng quát sau. `catch (Exception)` chỉ nên dùng ở tầng ngoài cùng (ví dụ middleware Web API ở Tập 3) để ghi log.
- `throw;` (không kèm biến) ném lại exception **giữ nguyên stack trace**. `throw ex;` làm mất stack trace gốc — tránh dùng.
- *Exception filter* `catch (X ex) when (điều kiện)`: chỉ bắt khi điều kiện đúng; nếu sai, exception tiếp tục đi lên như chưa bị bắt.

Khi nào ném exception?

- Tham số sai → `ArgumentException` / `ArgumentNullException` / `ArgumentOutOfRangeException`. Có hàm tiện ích: `ArgumentNullException.ThrowIfNull(x)`, `ArgumentOutOfRangeException.ThrowIfNegative(n)`.
- Trạng thái object không cho phép → `InvalidOperationException`.
- Lỗi "bình thường, dự đoán được" (người dùng nhập sai) → nên dùng mẫu `TryXxx` trả `bool`, vì exception tốn chi phí hơn.

using và IDisposable

```
using (var reader = new StreamReader(path)) { ... } // khối using
using var writer = new StreamWriter(path);         // using declaration: Dispose khi hết scope
```

Mọi class giữ tài nguyên hệ điều hành (file, socket, kết nối DB) implement `IDisposable`. `using` đảm bảo `Dispose()` được gọi **kể cả khi có exception** (compiler sinh `try/finally`).

API file hay dùng

Việc	API	Ghi chú
Đọc/ghi cả file nhỏ	<code>File.ReadAllText</code> , <code>File.WriteAllLines</code>	đơn giản nhất
Đọc từng dòng file lớn	<code>File.ReadLines</code> (lười), <code>StreamReader</code>	không nạp hết vào RAM
Ghi thêm	<code>File.AppendAllText</code> , <code>new StreamWriter(path, append: true)</code>	

Việc	API	Ghi chú
Đường dẫn	<code>Path.Combine</code> , <code>Path.GetFileName</code> , <code>Path.GetTempPath</code>	không tự nối bằng <code> + "/"</code>
Thư mục	<code>Directory.CreateDirectory</code> , <code>Directory.Delete(dir, true)</code>	
Thông tin	<code>FileInfo.Length</code> , <code>.Extension</code> , <code>File.Exists</code>	
Bất đồng bộ	<code>File.ReadAllTextAsync</code> , ...	dùng trong Web API (Tập 2 ch.5)

Lỗi và bẫy thường gặp

- **Nuốt lỗi** (`catch { }` rỗng): lỗi biến mất, rất khó debug. Ít nhất hãy ghi log.
- **Dùng exception cho luồng bình thường**: chậm và khó đọc. Dùng `TryParse`, `TryGetValue`.
- **Quên using**: file bị khóa, rò rỉ tài nguyên.
- **Nối đường dẫn bằng tay** `"dir" + "\\ " + "file"`: sai trên Linux. Dùng `Path.Combine`.
- **File.Exists rồi mới mở**: file có thể bị xóa giữa hai lệnh (race condition). Hãy mở và bắt `FileNotFoundException`.
- **Quên encoding**: nên chỉ rõ `Encoding.UTF8` khi làm việc với tiếng Việt.

Tóm tắt

- `try/catch/finally`, bắt exception cụ thể, dùng `when` khi cần lọc.
- Custom exception kế thừa `Exception`, thêm thuộc tính riêng.
- `using` đảm bảo tài nguyên được giải phóng.
- `File`, `Path`, `Directory`, `StreamReader/Writer` là bộ công cụ file cơ bản.

Bài tập (có lời giải)

1. Ghi file log 4 dòng rồi đếm số dòng bắt đầu bằng `ERROR`.
2. Viết `ReadOrNull(path)` trả về nội dung file hoặc `null` nếu file không tồn tại.
3. Tạo `InsufficientFundsException` và ném nó khi rút quá số dư; dùng `finally` để dọn thư mục tạm.

▼ Lời giải

```
using System.Text;

string dir = Path.Combine(Path.GetTempPath(), "csharp-book-ch06-sol");
Directory.CreateDirectory(dir);
string log = Path.Combine(dir, "app.log");

// Bài 1: ghi log và đếm số dòng ERROR
File.WriteAllLines(log, ["INFO start", "ERROR db timeout", "INFO retry", "ERROR db timeout"], Encoding.UTF8);
int errors = File.ReadLines(log).Count(l => l.StartsWith("ERROR"));
Console.WriteLine($"Bài 1: số dòng ERROR = {errors}");

// Bài 2: hàm đọc file an toàn, trả về null nếu không có file
Console.WriteLine($"Bài 2: có file -> {ReadOrNull(log)?.Length} ký tự; không có file -> {ReadOrNull(Path.Combine(dir, "x.txt")) ?? "null"}");
```

```
// Bài 3: custom exception + finally
try
{
    Withdraw(balance: 100, amount: 150);
}
catch (InsufficientFundsException ex)
{
    Console.WriteLine($"Bài 3: {ex.Message}");
}
finally
{
    Directory.Delete(dir, recursive: true);
    Console.WriteLine("Bài 3: finally đã dọn thư mục tạm");
}

static string? ReadOrNull(string path)
{
    try { return File.ReadAllText(path); }
    catch (FileNotFoundException) { return null; }
}

static void Withdraw(decimal balance, decimal amount)
{
    if (amount > balance) throw new InsufficientFundsException(amount - balance);
}

public class InsufficientFundsException(decimal missing)
    : Exception($"Thiếu {missing} để rút tiền")
{
    public decimal Missing { get; } = missing;
}
```

Bài 1: số dòng ERROR = 2
 Bài 2: có file -> 56 ký tự; không có file -> null
 Bài 3: Thiếu 50 để rút tiền
 Bài 3: finally đã dọn thư mục tạm

Bài 1 dùng `File.ReadLines` (đọc lười từng dòng) kết hợp `Count` của LINQ.

Nguồn tham khảo (Sources)

- Exceptions and exception handling: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/exceptions/index.md>
- Creating and throwing exceptions: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/exceptions/creating-and-throwing-exceptions.md>
- Exception handling (try/catch/finally): <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/exceptions/exception-handling.md>
- File and stream I/O: <https://github.com/dotnet/docs/blob/main/docs/standard/io/index.md>

Chương 7 — C# cho Java và TypeScript developer

Mục tiêu

- Chuyển nhanh kiến thức TypeScript và Java sang C# bằng bảng so sánh song song.
- Chạy **cùng một bài toán** bằng 3 ngôn ngữ và so sánh output.
- Biết những thói quen cần "quên đi" khi viết C#.

Giải thích đơn giản

Nếu bạn biết TypeScript và đang học Java, bạn đã biết khoảng 70% C#:

- **Cú pháp và OOP** gần như Java (class, interface, `new`, kiểu tĩnh, một class cha).
- **Tính năng hiện đại** gần như TypeScript (lambda `=>`, `?.`, `??`, `async/await`, type inference với `var`, string interpolation).
- **Điểm riêng của C#**: property, LINQ, `struct`, `record`, generics giữ kiểu lúc chạy, pattern matching, `ref/out`.

Ví dụ

Bài toán: danh sách user → lọc người lớn → in email nếu có → copy record → tìm user bất đồng bộ → xử lý lỗi.

C#

```
// Cùng một bài toán, viết bằng C#. So sánh với compare.ts và Compare.java.
List<User> users =
[
    new(1, "An", 28, "an@example.com"),
    new(2, "Bình", 17, null),
    new(3, "Chi", 35, "chi@example.com"),
];

// 1. Lọc + biến đổi (LINQ ~ filter/map trong TS, Stream trong Java)
var adults = users.Where(u => u.Age >= 18).Select(u => u.Name.ToUpper()).ToList();
Console.WriteLine($"Người lớn: {string.Join(" ", adults)}");

// 2. Null an toàn: ?. và ??
foreach (var u in users)
    Console.WriteLine($"{u.Name}: {u.Email?.Length.ToString() ?? "không có email"}");

// 3. Record: so sánh theo giá trị, copy bằng `with`
var older = users[0] with { Age = 29 };
var same = new User(1, "An", 28, "an@example.com");
Console.WriteLine(older);
Console.WriteLine($"users[0] == bản sao cùng dữ liệu? {users[0] == same}");

// 4. async/await
var name = await FindNameAsync(3);
Console.WriteLine($"Tìm thấy: {name}");

// 5. Exception
try
{
```

```

        await FindNameAsync(99);
    }
    catch (KeyNotFoundException ex)
    {
        Console.WriteLine($"Lỗi: {ex.Message}");
    }

    async Task<string> FindNameAsync(int id)
    {
        await Task.Delay(10); // giả lập gọi DB/HTTP
        var user = users.FirstOrDefault(u => u.Id == id)
            ?? throw new KeyNotFoundException($"User {id} không tồn tại");
        return user.Name;
    }

    record User(int Id, string Name, int Age, string? Email);

```

Người lớn: AN, CHI
 An: 14
 Bình: không có email
 Chi: 15
 User { Id = 1, Name = An, Age = 29, Email = an@example.com }
 users[0] == bản sao cùng dữ liệu? True
 Tìm thấy: Chi
 Lỗi: User 99 không tồn tại

TypeScript

```

// Same problem in TypeScript.
type User = { readonly id: number; readonly name: string; readonly age: number; readonly
email: string | null };

const users: User[] = [
    { id: 1, name: "An", age: 28, email: "an@example.com" },
    { id: 2, name: "Bình", age: 17, email: null },
    { id: 3, name: "Chi", age: 35, email: "chi@example.com" },
];

// 1. filter + map
const adults = users.filter(u => u.age >= 18).map(u => u.name.toUpperCase());
console.log(`Người lớn: ${adults.join(", ")}`);

// 2. ?. and ??
for (const u of users) console.log(`${u.name}: ${u.email?.length.toString()} ?? "không có
email"`)`);

// 3. spread copy (no built-in value equality)
const older = { ...users[0], age: 29 };
const same: User = { id: 1, name: "An", age: 28, email: "an@example.com" };
console.log(JSON.stringify(older));
console.log(`users[0] === bản sao cùng dữ liệu? ${users[0] === same}`);

async function findNameAsync(id: number): Promise<string> {
    await new Promise(r => setTimeout(r, 10));
    const user = users.find(u => u.id === id);
    if (!user) throw new Error(`User ${id} không tồn tại`);
    return user.name;
}

async function main() {
    // 4. async/await
    console.log(`Tìm thấy: ${await findNameAsync(3)}`);
    // 5. exceptions
    try {
        await findNameAsync(99);
    }
}

```

```

    } catch (e) {
        console.log(`Lỗi: ${e as Error}.message`);
    }
}
main();

```

```

$ tsc --version
Version 6.0.2
$ tsc --strict --target es2022 --module nodenext --outDir out compare.ts && node out/compare.js
Người lớn: AN, CHI
An: 14
Bình: không có email
Chi: 15
{"id":1,"name":"An","age":29,"email":"an@example.com"}
users[0] === bản sao cùng dữ liệu? false
Tìm thấy: Chi
Lỗi: User 99 không tồn tại

```

Java 21

```

// Same problem in Java 21.
import java.util.*;
import java.util.concurrent.*;

public class Compare {
    record User(int id, String name, int age, String email) {}

    static final List<User> users = List.of(
        new User(1, "An", 28, "an@example.com"),
        new User(2, "Bình", 17, null),
        new User(3, "Chi", 35, "chi@example.com"));

    static CompletableFuture<String> findNameAsync(int id) {
        return CompletableFuture.supplyAsync(() -> {
            try { Thread.sleep(10); } catch (InterruptedException e) { throw new
RuntimeException(e); }
            return users.stream().filter(u -> u.id() == id).findFirst()
                .orElseThrow(() -> new NoSuchElementException("User " + id + " không tồn
tại"))
                .name();
        });
    }

    public static void main(String[] args) {
        // 1. Stream filter + map
        var adults = users.stream().filter(u -> u.age() >= 18).map(u ->
u.name().toUpperCase()).toList();
        System.out.println("Người lớn: " + String.join(", ", adults));

        // 2. Null: Optional (Java has no ?. operator)
        for (var u : users)
            System.out.println(u.name() + ": " + Optional.ofNullable(u.email()).map(e ->
String.valueOf(e.length())).orElse("không có email"));

        // 3. Record: value equality, but no `with` (make a new one)
        var first = users.get(0);
        var older = new User(first.id(), first.name(), 29, first.email());
        var same = new User(1, "An", 28, "an@example.com");
        System.out.println(older);
        System.out.println("users[0].equals(bản sao cùng dữ liệu)? " + first.equals(same));

        // 4. async (CompletableFuture) - join() blocks here
        System.out.println("Tìm thấy: " + findNameAsync(3).join());

        // 5. Exception
    }
}

```

```

    try {
        findNameAsync(99).join();
    } catch (CompletionException ex) {
        System.out.println("Lỗi: " + ex.getCause().getMessage());
    }
}

```

```

$ java --version
openjdk 21.0.10 2026-01-20
$ java Compare.java
Người lớn: AN, CHI
An: 14
Bình: không có email
Chi: 15
User[id=1, name=An, age=29, email=an@example.com]
users[0].equals(bản sao cùng dữ liệu)? true
Tìm thấy: Chi
Lỗi: User 99 không tồn tại

```

Kết quả giống nhau, trừ dòng so sánh bằng nhau: C# `record` và Java `record` so sánh theo **giá trị** (True/true), còn object TypeScript so sánh theo **tham chiếu** (false).

Đi sâu — bảng so sánh song song

1. Công cụ và project

Việc	TypeScript	Java	C#
Runtime	Node.js	JVM	.NET Runtime (CLR)
Biên dịch	tsc → JS	javac → bytecode	Roslyn → IL
File project	package.json, tsconfig.json	pom.xml / build.gradle	*.csproj (+ Directory.Build.props)
Kho gói	npm	Maven Central	NuGet
Thêm gói	npm i zod	sửa pom.xml	dotnet add package X
Chạy một file	node a.js	java A.java	dotnet run a.cs (.NET 10)
Test	Jest / Vitest	JUnit	xUnit / NUnit / MSTest

2. Kiểu dữ liệu

Khái niệm	TypeScript	Java	C#
Số nguyên	number	int, long	int, long
Số thực	number	double	double, decimal (tiền)
Chuỗi	string	String	string (= System.String)
Suy luận kiểu	let x = 1	var x = 1;	var x = 1;
Hằng	const	final	const (lúc biên dịch), readonly (field)
Any	any / unknown	Object	object (hoặc dynamic, hiếm dùng)
Union	'a' 'b'	sealed interface + record	enum, hoặc class hierarchy (union thật: C# 15, preview)

Khái niệm	TypeScript	Java	C#
Tuple	[number, string]	không có sẵn	(int, string)
Kiểu tự định nghĩa value type	không	không (Java 21)	struct

3. Null

	TypeScript	Java	C#
Khai báo có thể null	string null	mọi object	string?
Truy cập an toàn	a?.b	Optional.map	a?.b
Giá trị mặc định	a ?? b	Optional.orElse	a ?? b
Gán nếu null	a ??= b	—	a ??= b
Gán có điều kiện	—	—	a?.b = c (C# 14)
"Tôi chắc không null"	a!	—	a!

4. Class và property

	TypeScript	Java	C#
Property	get x() / field public	getter/setter	public int X { get; set; }
Chỉ đọc	readonly x	final field + getter	{ get; } hoặc { get; init; }
Constructor ngắn	constructor(private x: number)	record	primary constructor class A(int x)
Override	tự động	tự động (@Override)	phải virtual + override
Không cho kế thừa	—	final class	sealed class
Extension method	—	—	static method với this hoặc khối extension (C# 14)

5. Collections và xử lý dữ liệu

	TypeScript	Java	C# (LINQ)
Lọc	arr.filter(f)	stream().filter(f)	list.Where(f)
Biến đổi	arr.map(f)	.map(f)	.Select(f)
Gộp	arr.reduce(f, 0)	.reduce(0, f)	.Aggregate(0, f) / .Sum()
Tìm	arr.find(f)	.filter(f).findFirst()	.FirstOrDefault(f)
Có phần tử?	arr.some(f)	.anyMatch(f)	.Any(f)
Nhóm	Object.groupBy	Collectors.groupingBy	.GroupBy(f)
Kết quả	mảng (ngay)	stream (lười)	IEnumerable (lười) → .ToList()

6. Bất đồng bộ

	TypeScript	Java	C#
Kiểu	<code>Promise<T></code>	<code>CompletableFuture<T></code>	<code>Task<T></code>
Hàm	<code>async function</code>	—	<code>async Task<T> M()</code>
Chờ	<code>await p</code>	<code>.join()</code> / <code>.get()</code> (block)	<code>await t</code>
Chờ nhiều	<code>Promise.all</code>	<code>CompletableFuture.allOf</code>	<code>Task.WhenAll</code>
Hủy	<code>AbortController</code>	<code>cancel()</code>	<code>CancellationToken</code>
Luồng	1 luồng + event loop	nhiều thread / virtual threads	thread pool; <code>await</code> không chặn thread

7. Generics

- TypeScript: chỉ tồn tại lúc biên dịch, **structural** (so theo hình dạng).
- Java: *type erasure* — lúc chạy `List<String>` và `List<Integer>` là cùng một kiểu; không dùng được `List<int>`.
- C#: *reified* — lúc chạy vẫn biết `List<int>` khác `List<string>`; `typeof(T)` dùng được; `List<int>` chứa `int` thật, không boxing.

8. Module và đặt tên

	TypeScript	Java	C#
Nhóm code	<code>module (import/export)</code>	<code>package</code>	<code>namespace</code>
Import	<code>import { A } from './a'</code>	<code>import a.b.A;</code>	<code>using A.B;</code>
Phạm vi mặc định	<code>file</code>	<code>package-private</code>	<code>internal</code> (trong assembly)
Tên method	<code>camelCase</code>	<code>camelCase</code>	PascalCase
Tên property	<code>camelCase</code>	<code>getX()</code>	PascalCase
Field private	<code>#x</code> / <code>private x</code>	<code>x</code>	<code>_x</code>
Interface	<code>User</code>	<code>User</code>	IUser (tiền tố I)

9. Lỗi

	TypeScript	Java	C#
Checked exception	không	có (<code>throws</code>)	không
Bắt theo kiểu	không (<code>catch (e)</code>)	có	có, thêm <code>when</code> (điều kiện)
Giải phóng tài nguyên	<code>try/finally</code>	<code>try-with-resources</code>	<code>using</code>

Lỗi và bẫy thường gặp (thói quen cần quên)

- **Từ TypeScript:** `==` trong C# không ép kiểu như JS; không có `===`. Chia `int` cho `int` ra số nguyên. Object literal `{ a: 1 }` không có — dùng `new { A = 1 }` (anonymous type) hoặc record.
- **Từ TypeScript:** kiểu trong C# tồn tại lúc chạy; `obj is User` u kiểm tra kiểu thật.

- **Từ Java:** method không virtual mặc định; tên method viết hoa chữ đầu; `string` so sánh bằng `==` là **đúng** trong C# (khác `equals` của Java).
- **Từ Java:** không cần getter/setter thủ công — dùng property.
- **Cả hai:** không gọi `.Result` / `.Wait()` trên `Task` (để deadlock/chặn thread); dùng `await` từ đầu đến cuối ("async all the way").
- **Cả hai:** LINQ lười (lazy) giống Java stream: query chỉ chạy khi duyệt hoặc gọi `.ToList()`.

Tóm tắt

- C# = cú pháp và OOP kiểu Java + tính năng hiện đại kiểu TypeScript.
- Cần học thêm: property, LINQ, `record`, `struct`, pattern matching, `Task/await`, quy tắc đặt tên PascalCase.
- Tập 2 đi sâu vào generics, LINQ, records, async — những thứ bảng trên chỉ giới thiệu.

Bài tập (có lời giải)

1. Dịch đoạn TypeScript sau sang C#: `const total = orders.filter(o => o.paid).reduce((s, o) => s + o.amount, 0);`
2. Viết lại hàm Java trả về `Optional<String>` thành hàm C# trả về `string?`, và dùng `??` khi gọi.
3. TypeScript có `type Status = 'ok' | 'error'`. Viết tương đương trong C# bằng `enum` và `switch expression`.

▼ Lời giải

```
// Bài 1: dịch đoạn TypeScript sau sang C#:
// const total = orders.filter(o => o.paid).reduce((s, o) => s + o.amount, 0);
Order[] orders = [new(1, 100m, true), new(2, 50m, false), new(3, 70m, true)];
decimal total = orders.Where(o => o.Paid).Sum(o => o.Amount);
Console.WriteLine($"Bài 1: tổng đã thanh toán = {total}");

// Bài 2: Java Optional<String> -> C# nullable
string? FindEmail(int id) => id == 1 ? "an@example.com" : null;
Console.WriteLine($"Bài 2: {FindEmail(1)} ?? "(none)" | {FindEmail(2)} ?? "(none)"}");

// Bài 3: TS union type 'ok' | 'error' -> C# enum + switch expression
foreach (var st in Enum.GetValues<Status>())
    Console.WriteLine($"Bài 3: {st} -> {Describe(st)}");

static string Describe(Status s) => s switch
{
    Status.Ok => "thành công",
    Status.Error => "thất bại",
    _ => throw new ArgumentOutOfRangeException(nameof(s)),
};

record Order(int Id, decimal Amount, bool Paid);
enum Status { Ok, Error }
```

```
Bài 1: tổng đã thanh toán = 170
Bài 2: an@example.com | (none)
Bài 3: Ok -> thành công
Bài 3: Error -> thất bại
```

Bài 1: `Where = filter`, `Sum(selector)` thay cho `reduce` khi chỉ cộng dồn. Bài 3: nhánh `_ => throw` ... bảo vệ trường hợp giá trị enum không hợp lệ (enum trong C# thực chất là số nguyên, có thể ép kiểu từ số bất kỳ).

Nguồn tham khảo (Sources)

- Tour of C# (nguồn Microsoft Learn): <https://github.com/dotnet/docs/blob/main/docs/csharp/tour-of-csharp/overview.md>
- Tips for Java developers: <https://github.com/dotnet/docs/blob/main/docs/csharp/tour-of-csharp/tips-for-java-developers.md>
- Tips for JavaScript/TypeScript developers: <https://github.com/dotnet/docs/blob/main/docs/csharp/tour-of-csharp/tips-for-javascript-developers.md>
- C# identifier naming rules and conventions: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/coding-style/identifier-names.md>
- Null-conditional assignment (C# 14 proposal): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-14.0/null-conditional-assignment.md>
- Unions (C# 15 proposal): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-15.0/unions.md>

GLOSSARY — Thuật ngữ (Tiếng Anh → Tiếng Việt)

Sắp xếp theo thứ tự chữ cái. Cột "Tập" cho biết thuật ngữ xuất hiện lần đầu ở đâu.

Thuật ngữ	Giải thích tiếng Việt	Tập
abstract class	class trừu tượng, không tạo object trực tiếp, dùng làm lớp cha	1
access modifier	từ khóa phạm vi truy cập: <code>public</code> , <code>private</code> , <code>protected</code> , <code>internal</code>	1
ArrayPool	kho mảng dùng lại, giảm cấp phát bộ nhớ	3
assembly	file <code>.dll</code> / <code>.exe</code> kết quả build một project	1
async / await	cú pháp viết code bất đồng bộ như code tuần tự	1, 2
back-pressure	cơ chế làm bên gửi chậm lại khi bên nhận xử lý không kịp	3
backing field	field ẩn chứa giá trị thật của property	1
BenchmarkDotNet	thư viện đo hiệu năng chuẩn của .NET	3
boxing	đóng gói value type vào object trên heap (tốn bộ nhớ)	2
Channel<T>	hàng đợi bất đồng bộ cho mẫu producer/consumer	3
checked / unchecked	bật/tắt kiểm tra tràn số	1
clean architecture	kiến trúc chia tầng, phụ thuộc chỉ hướng vào trong	3
closure	lambda "nhớ" biến ở phạm vi bên ngoài	2
collection expression	cú pháp <code>[1, 2, 3]</code> tạo collection; <code>..</code> là spread	1
collection	tập hợp nhiều phần tử: <code>List<T></code> , <code>Dictionary</code> , ...	1
constraint (generic)	ràng buộc <code>where T : ...</code> cho tham số kiểu	2
constructor	hàm khởi tạo object	1
container / image (Docker)	image: gói ứng dụng + môi trường; container: image đang chạy	3
DbContext	phiên làm việc với database trong EF Core	3
deadlock	hai luồng chờ nhau mãi mãi	3
decimal	kiểu số thập phân 128 bit, dùng cho tiền	1
deconstruction	tách một giá trị thành nhiều biến: <code>var (a, b) = tuple;</code>	1
deferred execution	thực thi trì hoãn: query LINQ chỉ chạy khi duyệt	2
delegate	kiểu đại diện cho một hàm (con trỏ hàm an toàn kiểu)	2
dependency injection (DI)	tiêm phụ thuộc: object nhận dependency từ bên ngoài thay vì tự tạo	2
Dispose / IDisposable	giải phóng tài nguyên (file, kết nối)	1
DTO (Data Transfer Object)	object chỉ chở dữ liệu request/response	2, 3
EF Core	Entity Framework Core — ORM của .NET	3
endpoint filter	bộ lọc chạy quanh một endpoint Minimal API	3
event	cơ chế thông báo dựa trên delegate	2
exception filter	<code>catch (X) when (...)</code> : chỉ bắt khi điều kiện đúng	1
exception	ngoại lệ, object mô tả lỗi lúc chạy	1

Thuật ngữ	Giải thích tiếng Việt	Tập
extension member	thêm method/property cho kiểu có sẵn mà không sửa kiểu đó (C# 14 mở rộng)	1, 2
field keyword	từ khóa C# 14 truy cập backing field trong property	1
file-based app	ứng dụng chỉ một file <code>.cs</code> , chạy bằng <code>dotnet run app.cs</code> (.NET 10)	1
fixture (test)	dữ liệu/tài nguyên dùng chung giữa các test	2
GC (Garbage Collector)	bộ gom rác, tự giải phóng object không còn dùng	3
Generic Host	"khung" ứng dụng gồm DI, configuration, logging	2
generic	kiểu/hàm có tham số kiểu <code><T></code>	1, 2
health check	endpoint báo ứng dụng còn sống / sẵn sàng	3
heap / stack	vùng nhớ cho object (heap) và cho biến cục bộ, lời gọi hàm (stack)	1
IL (Intermediate Language)	ngôn ngữ trung gian mà C# biên dịch ra	1
immutable	bất biến, không sửa được sau khi tạo	1
init accessor	<code>init</code> : chỉ gán lúc khởi tạo object	1
integration test	test nhiều thành phần cùng chạy thật (HTTP, DB)	3
interface	"hợp đồng" gồm các thành viên mà class phải cài đặt	1
Interlocked	thao tác nguyên tử trên biến số, an toàn đa luồng	3
JIT (Just-In-Time)	biên dịch IL sang mã máy lúc chạy	1
lifetime (DI)	vòng đời service: Singleton, Scoped, Transient	2
LINQ	Language Integrated Query: truy vấn dữ liệu bằng <code>Where</code> , <code>Select</code> ...	1, 2
LTS / STS	Long/Standard Term Support: hỗ trợ dài hạn (3 năm) / ngắn hạn	1
Microsoft.Testing.Platform (MTP)	nền tảng chạy test mới, dùng với xUnit v3 trên .NET 10	2
middleware	thành phần xử lý request/response theo chuỗi	3
migration (EF Core)	file mô tả thay đổi schema database theo thời gian	3
Minimal API	cách viết Web API bằng lambda <code>MapGet/MapPost...</code>	3
namespace	không gian tên, nhóm các kiểu	1
NuGet	kho và trình quản lý gói của .NET	1
null-coalescing	toán tử <code>??</code> và <code>??=</code>	1
null-conditional	toán tử <code>?.</code> và <code>?[]</code>	1
nullable	có thể mang giá trị <code>null</code> : <code>int?</code> , <code>string?</code>	1
observability	khả năng quan sát hệ thống qua logs, traces, metrics	3
OpenTelemetry	chuẩn mở thu thập và xuất logs/traces/metrics	3
ORM	ánh xạ object ↔ bảng database	3
override / virtual	ghi đè method của lớp cha / cho phép ghi đè	1
pattern matching	so khớp mẫu: <code>is</code> , <code>switch</code> với pattern	1, 2
primary constructor	tham số constructor khai báo ngay sau tên class	1
Problem Details	định dạng JSON chuẩn cho lỗi HTTP (RFC 7807, bản mới RFC 9457)	3

Thuật ngữ	Giải thích tiếng Việt	Tập
property	thuộc tính có <code>get / set</code>	1
race condition	kết quả sai do nhiều luồng cùng sửa dữ liệu	3
record	kiểu dữ liệu so sánh theo giá trị, hỗ trợ <code>with</code>	1, 2
reference type	kiểu tham chiếu: gán là copy địa chỉ	1
reified generics	generic giữ thông tin kiểu lúc chạy (khác type erasure của Java)	2
required member	thành viên bắt buộc gán khi khởi tạo	1
runtime	môi trường chạy chương trình (.NET Runtime / CLR)	1
scope (logging)	ngữ cảnh gắn thêm trường cho mọi log bên trong	3
SDK	bộ công cụ phát triển (<code>dotnet CLI, compiler</code>)	1
sealed	không cho kế thừa tiếp	1
<code>Span<T></code>	"cửa sổ" nhìn vào vùng nhớ liên tục, không copy	3
<code>stackalloc</code>	cấp phát bộ nhớ tạm trên stack	3
<code>struct</code>	kiểu giá trị do người dùng định nghĩa	1
structured logging	log có cấu trúc: giữ tên và giá trị từng trường	2, 3
switch expression	<code>x switch { pattern => value, ... }</code> trả về giá trị	1
test double / fake / mock	đối tượng giả thay dependency thật khi test	2
Theory / Fact (xUnit)	test có dữ liệu tham số / test đơn	2
top-level statements	viết code trực tiếp trong <code>Program.cs</code> , không cần <code>Main</code>	1
trace / span	hành trình của một request / một bước trong hành trình đó	3
tuple	nhóm nhiều giá trị: <code>(int Min, int Max)</code>	1
use case	trường hợp sử dụng: một thao tác nghiệp vụ của ứng dụng	3
value type	kiểu giá trị: gán là copy dữ liệu	1
variance (in/out)	quy tắc gán generic giữa kiểu cha/con	2